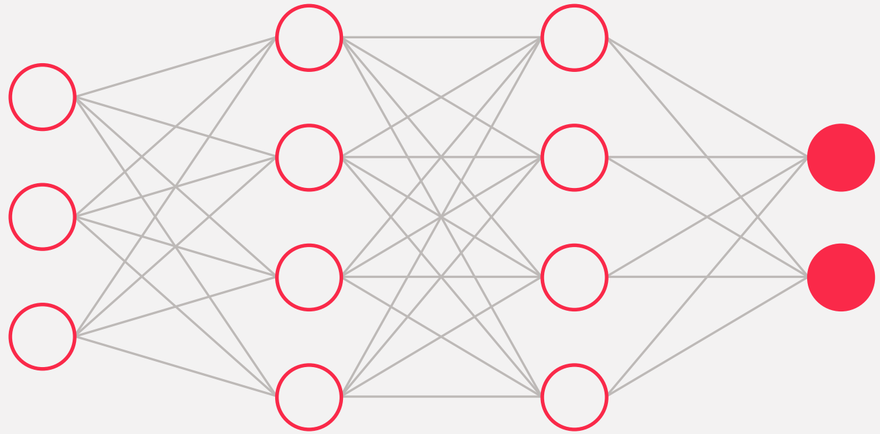
Cheat Sheet de Machine Learning

Conceptos prácticos y transversales a todo el curso.



Contenido

01 Sesgo y varianza (bias-variance tradeoff)

02 Datos desbalanceados y métricas de clasificación

03 Reducción de dimensionalidad con PCA

04 Teorema de Bayes y clasificador Naïve Bayes

05 Análisis de regresión

06 Regularización en ML

07 Redes neuronales convolucionales: bloques básicos

08 CNNs famosas

09 Aprendizaje por ensamble

10 Estructuras de datos

01

Sesgo y varianza (bias-variance tradeoff)

01

Sesgo y varianza

Sesgo (bias)

Error entre la predicción promedio del modelo y el valor real: mide qué tan bien representa la relación real.

Alto sesgo → **Modelo demasiado simple** → **Subajuste**

Error alto en entrenamiento y en prueba

Varianza

Cuánto cambia la predicción entre distintos conjuntos de entrenamiento: mide su sensibilidad a los datos.

Alta varianza → **Modelo demasiado complejo** → **Sobreajuste**

Error bajo en entrenamiento, alto en prueba

Error total = Sesgo² + Varianza + Error irreducible

El mejor modelo es el que minimiza esta suma.

El tablero de dardos

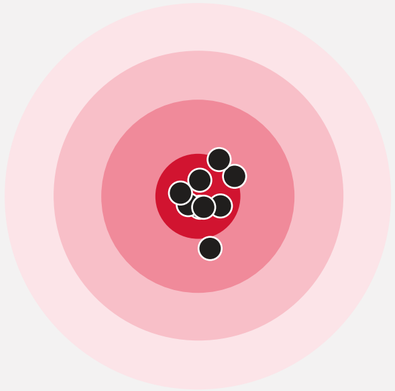
Cada panel es un modelo distinto; el centro es el valor real.

Sesgo = qué tan lejos del centro está el grupo de predicciones.

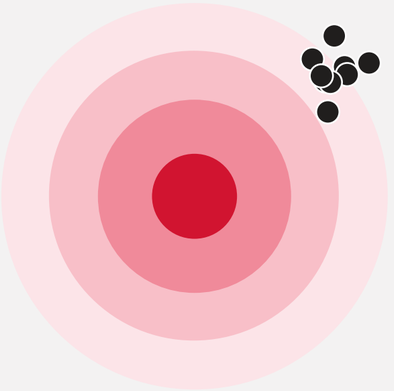
Varianza = qué tan dispersas están las predicciones entre sí.

Panel ideal: arriba-izquierda: cerca del centro y poco disperso.

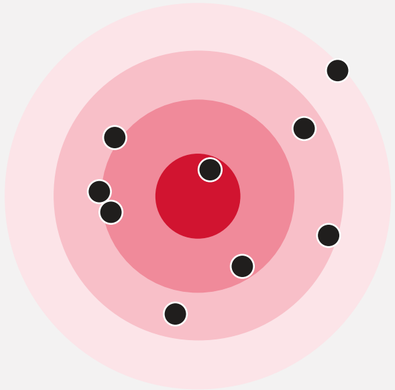
Sesgo bajo, varianza baja



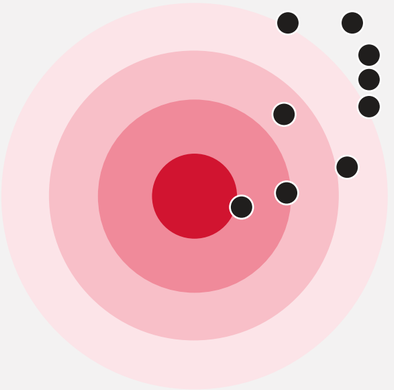
Sesgo alto, varianza baja



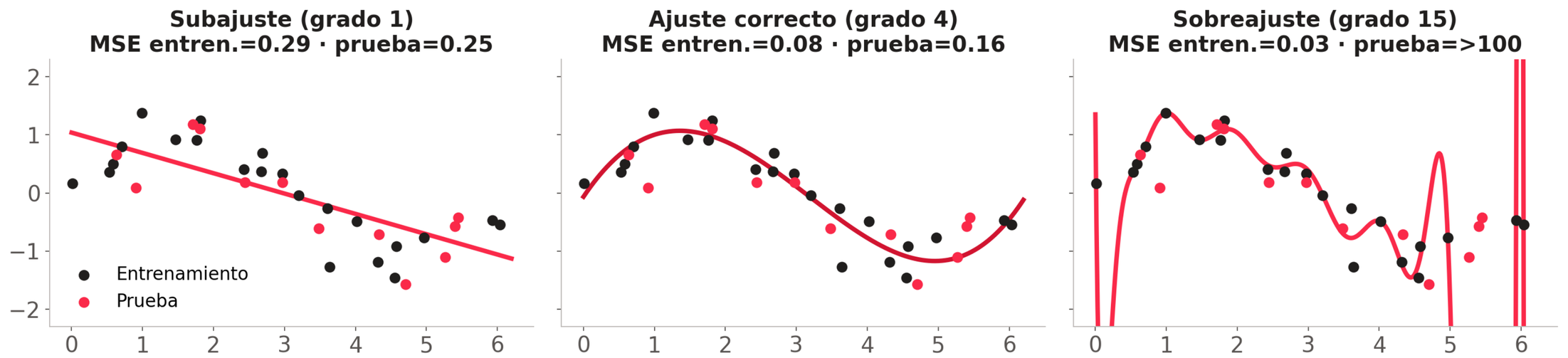
Sesgo bajo, varianza alta



Sesgo alto, varianza alta



Subajuste, ajuste correcto y sobreajuste



Subajuste · grado 1

No logra capturar la curva → error alto en entrenamiento y en prueba.

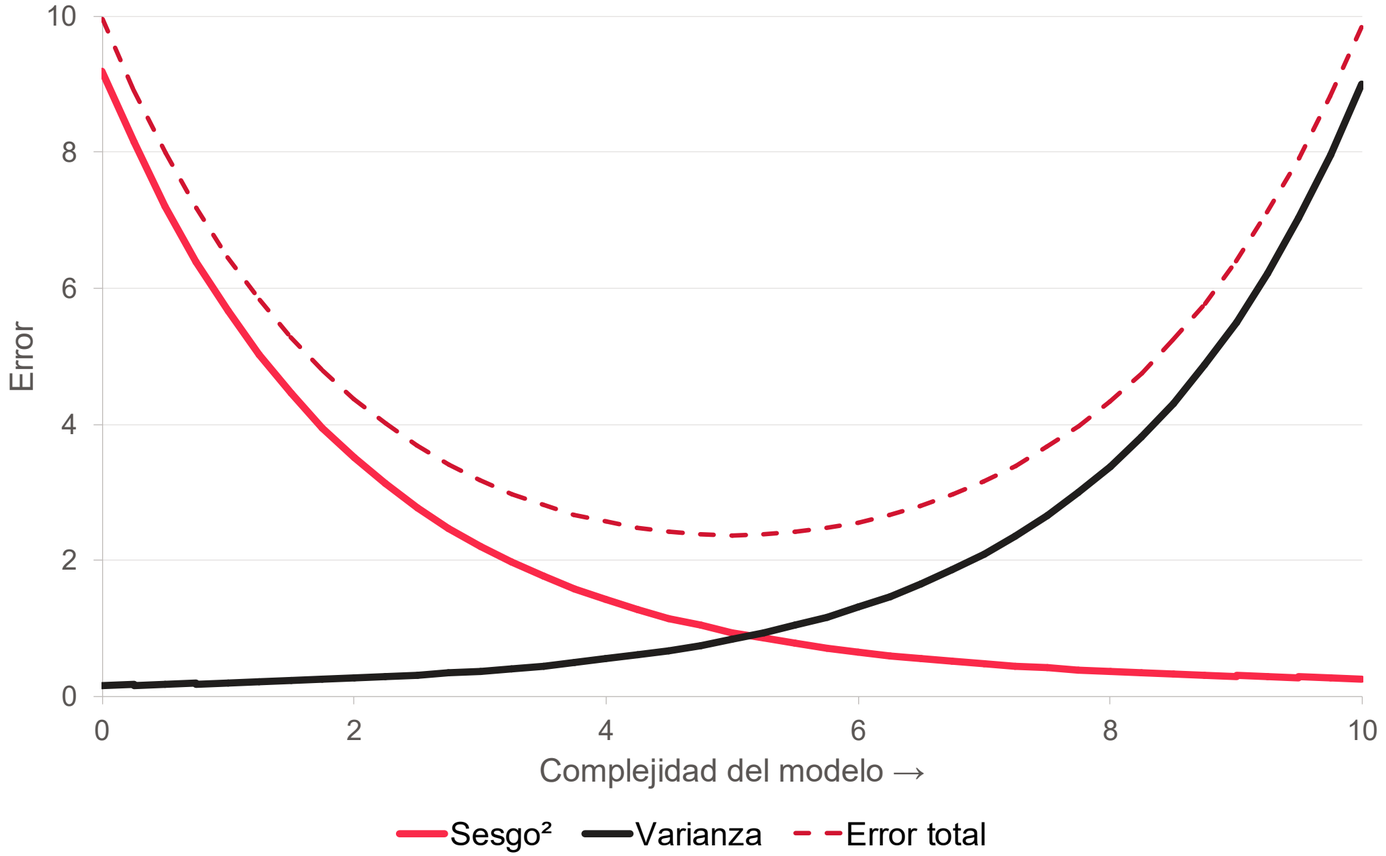
Ajuste correcto · grado 4

El mejor balance: error bajo y similar en ambos conjuntos.

Sobreajuste · grado 15

Casi toca cada punto de entrenamiento, pero oscila entre ellos → error de prueba mucho mayor.

El equilibrio sesgo–varianza



Subajuste

Poca complejidad: el sesgo domina.

Punto óptimo

Mínimo del error total: mejor generalización.

Sobreajuste

Mucha complejidad: la varianza domina.

Curvas ilustrativas (no corresponden a un conjunto de datos real).

02

Datos desbalanceados y métricas de clasificación

02

Más allá de la exactitud (accuracy)

Un clasificador que siempre predice la clase mayoritaria puede tener accuracy alta sin haber aprendido nada.

	Predicho +	Predicho −
Real +	**TP** Verdadero positivo	**FN** Falso negativo
Real −	**FP** Falso positivo	**TN** Verdadero negativo

Exactitud — **(TP+TN) / Total**

¿Qué % total acerté?

Precisión — **TP / (TP+FP)**

De lo que predije positivo, ¿cuánto era real?

Recall / Sensibilidad — **TP / (TP+FN)**

De los positivos reales, ¿cuántos detecté?

Especificidad — **TN / (TN+FP)**

De los negativos reales, ¿cuántos identifiqué?

F1 Score — **2·(Prec·Rec)/(Prec+Rec)**

Media armónica entre precisión y recall

Cómo tratar el desbalance de clases

1

Replicar datos

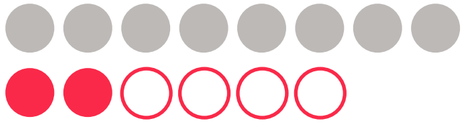
Duplicar ejemplos de la clase minoritaria hasta equilibrar.



2

Datos sintéticos (SMOTE)

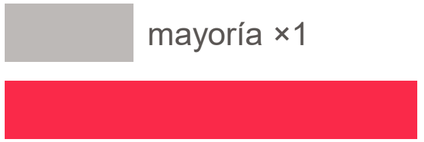
Generar ejemplos nuevos interpolando entre vecinos minoritarios.



3

Ponderar la pérdida

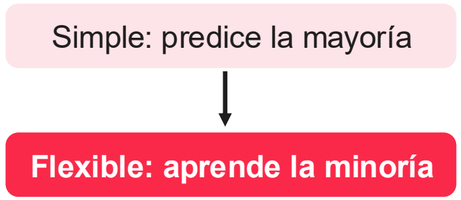
Con class_weight, un error en la minoría pesa más.



4

Aumentar la complejidad

Un modelo más flexible puede aprender también la clase minoritaria.



Después de balancear, evalúa con precisión, recall y F1 — no solo con la exactitud.

Clasificador ingenuo vs. modelo real

Clasificador ingenuo

siempre predice la clase mayoritaria

	Pred. 0	Pred. 1
Real 0	570	0
Real 1	30	0

Exactitud ≈ 95%

Recall clase minoritaria = 0

Regresión logística

modelo entrenado sin ajustes

	Pred. 0	Pred. 1
Real 0	568	2
Real 1	30	0

Exactitud ≈ 95%

Recall clase minoritaria = 0

La exactitud sola puede ocultar que un modelo no aprendió nada útil sobre la clase que más importa.

03

Reducción de dimensionalidad con PCA

03

¿Qué es PCA?

Busca un nuevo conjunto de ejes ortogonales donde la varianza es máxima y los ordena de mayor a menor (mayor varianza = más información).

1

Estandarizar

Cada variable con media 0 y desviación 1.

→

2

Matriz de covarianza

Mide cómo varían juntas las variables.

→

3

Autovalores y autovectores

Direcciones de los ejes y varianza que capturan.

→

4

Ordenar

De mayor a menor varianza.

→

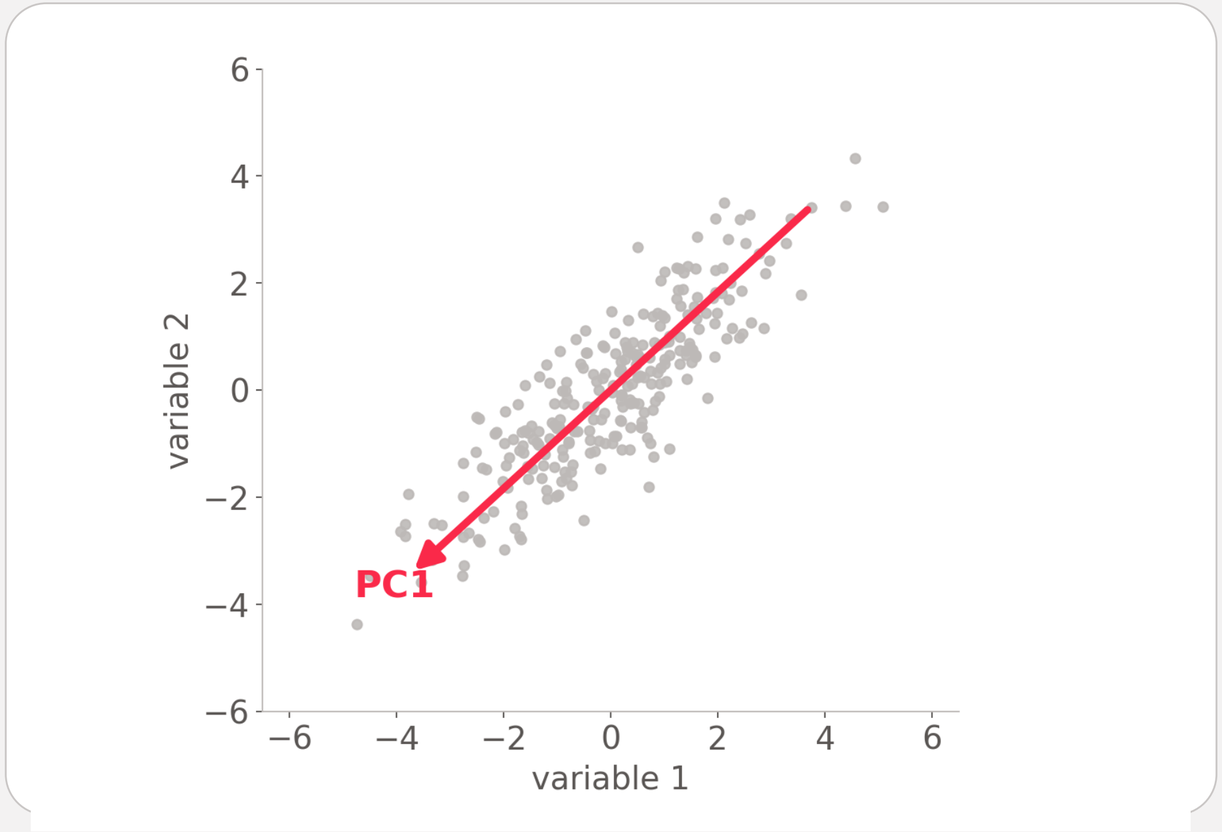
5

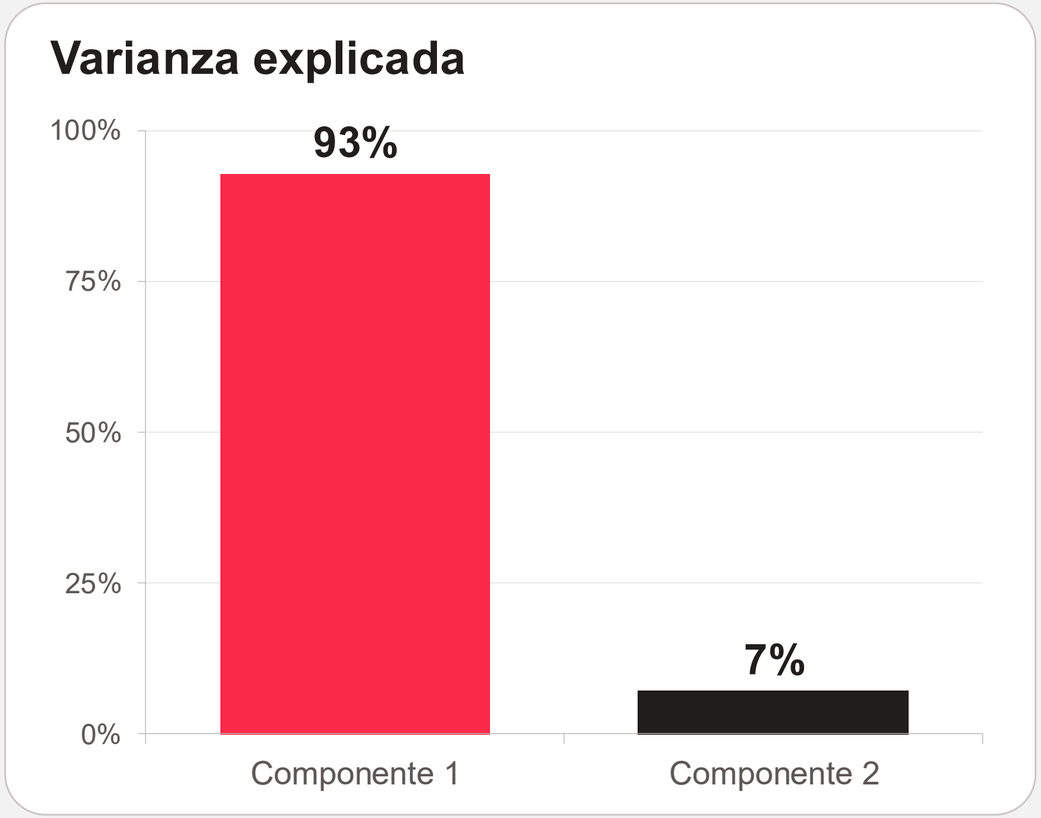
Conservar m ejes

Solo los primeros: los de mayor varianza.

Reducir dimensionalidad = conservar solo los primeros m ejes (los de mayor varianza).

PCA en acción

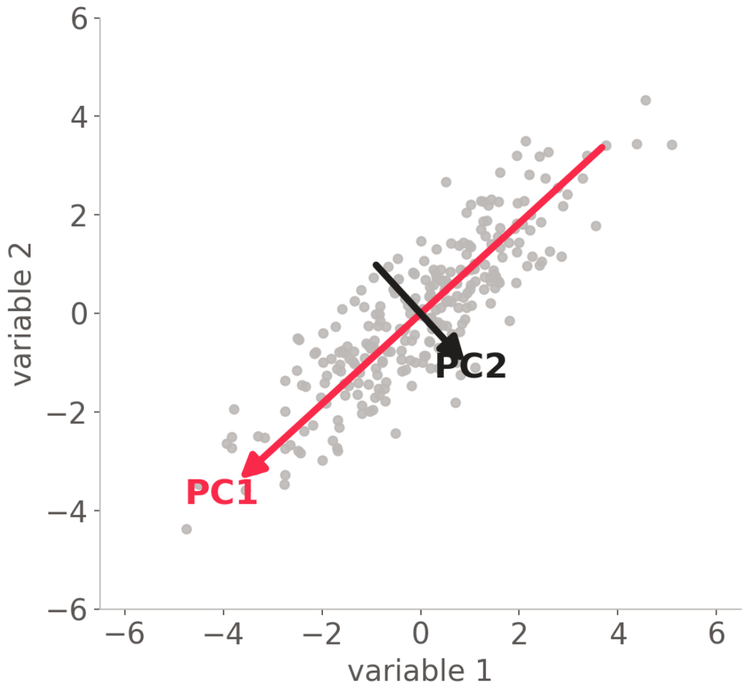




Quedarse solo con la componente 1 reduce a 1 dimensión conservando la mayoría de la información original.

Cada componente es un nuevo eje de los datos

Un componente combina las variables originales con distintos pesos; no es una variable original seleccionada.



PC1 Captura la mayor varianza posible: es la dirección principal de dispersión.

PC2 Es perpendicular a PC1 y captura la mayor varianza que queda.

PC3, … Siguen el mismo criterio: son ortogonales a los anteriores y explican cada vez menos varianza.

Pesos del eje ≠ puntuaciones: los pesos definen la combinación; las puntuaciones son las coordenadas de cada observación al proyectarla.

Los componentes se convierten en nuevas variables

1

Elegir cuántos conservar

Revisar la varianza explicada acumulada y retener k componentes según la pérdida de información aceptable.

2

Proyectar las observaciones

Multiplicar los datos centrados o estandarizados por los k ejes: Z = X_preparada · W_k. El resultado tiene k columnas.

3

Usar la representación reducida

Usar Z para visualizar en 2D/3D, agrupar datos o entrenar un modelo; evaluar si la reducción ayuda a la tarea.

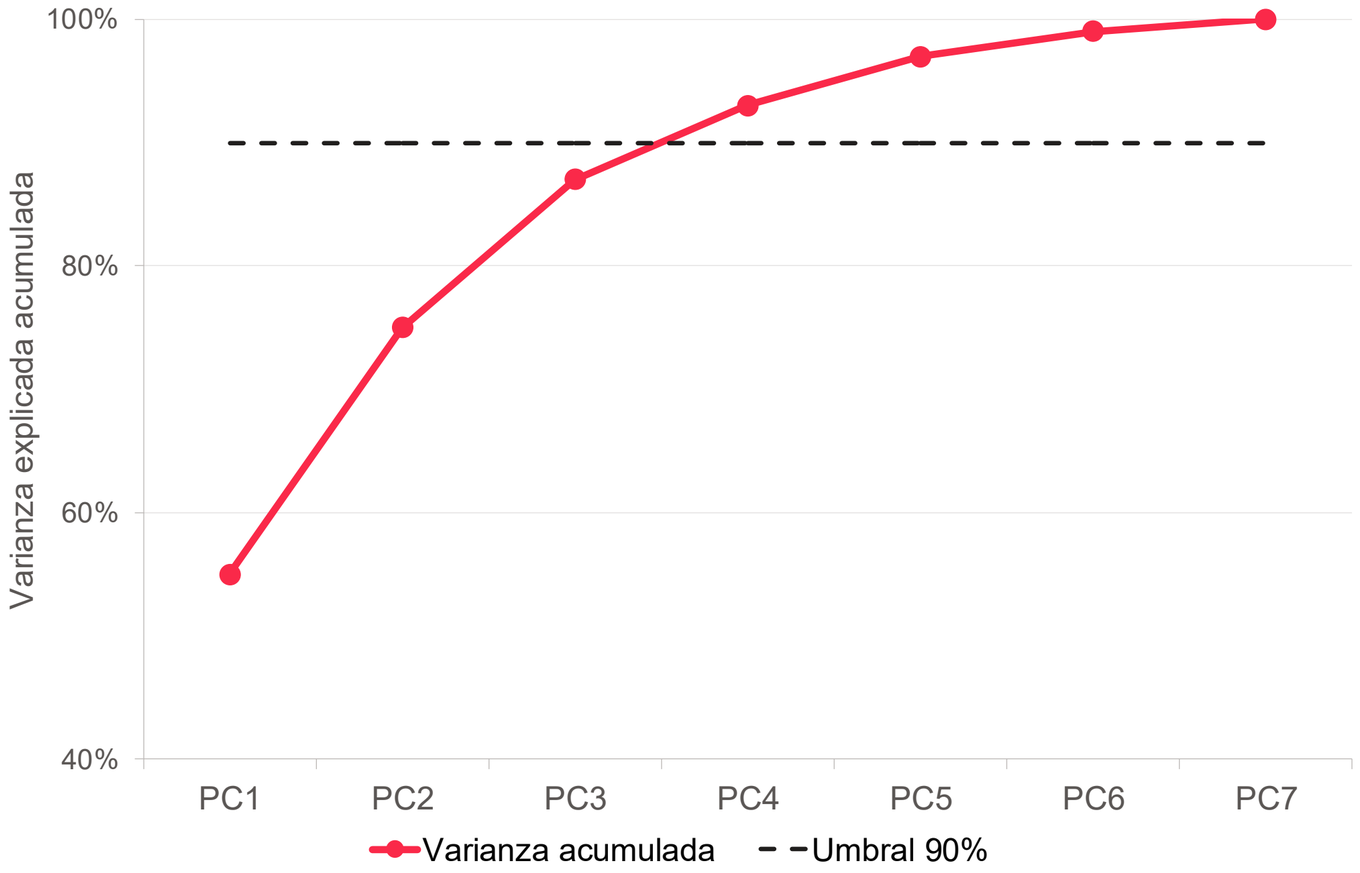
4

Transformar datos nuevos

Ajustar el preprocesamiento y PCA solo con entrenamiento. En validación y prueba, reutilizar ese mismo ajuste.

Reducir dimensiones conserva variación, pero no garantiza mejorar la predicción.

¿Cuántos componentes conservar?



Lee la curva

La varianza acumulada sube rápido y luego se aplana.

Elige k

Retén los componentes hasta alcanzar la pérdida de información aceptable (p. ej. 90%).

Valida

Comprueba si la reducción ayuda a la tarea.

Datos ilustrativos.

04

Teorema de Bayes y Clasificador Naïve Bayes

04

Teorema de Bayes

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

posterior = verosimilitud × prior / evidencia

P(A|B)

Posterior

Creencia actualizada al observar B

P(B|A)

Verosimilitud

Qué tan probable es B si A es cierto

P(A)

Prior

Creencia inicial sobre A

P(B)

Evidencia

Probabilidad total de observar B

MAP (máximo a posteriori)

Incorpora un prior: maximiza $P(y) \cdot \prod P(x_i|y)$.

MLE (máxima verosimilitud)

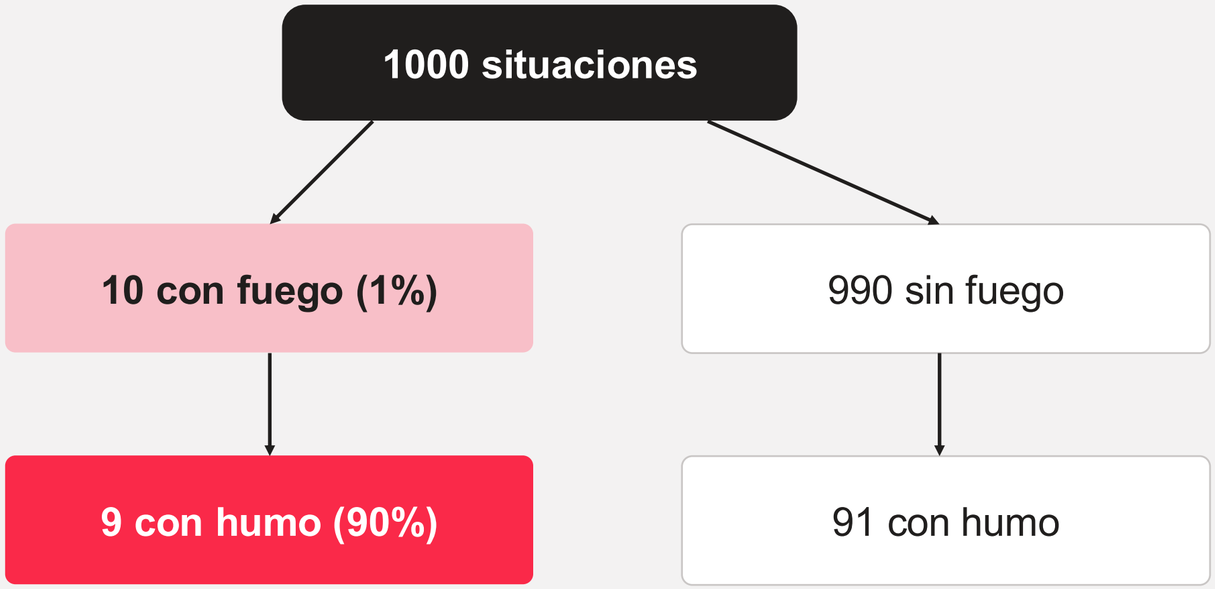
Asume prior uniforme: maximiza solo $\prod P(x_i|y)$.

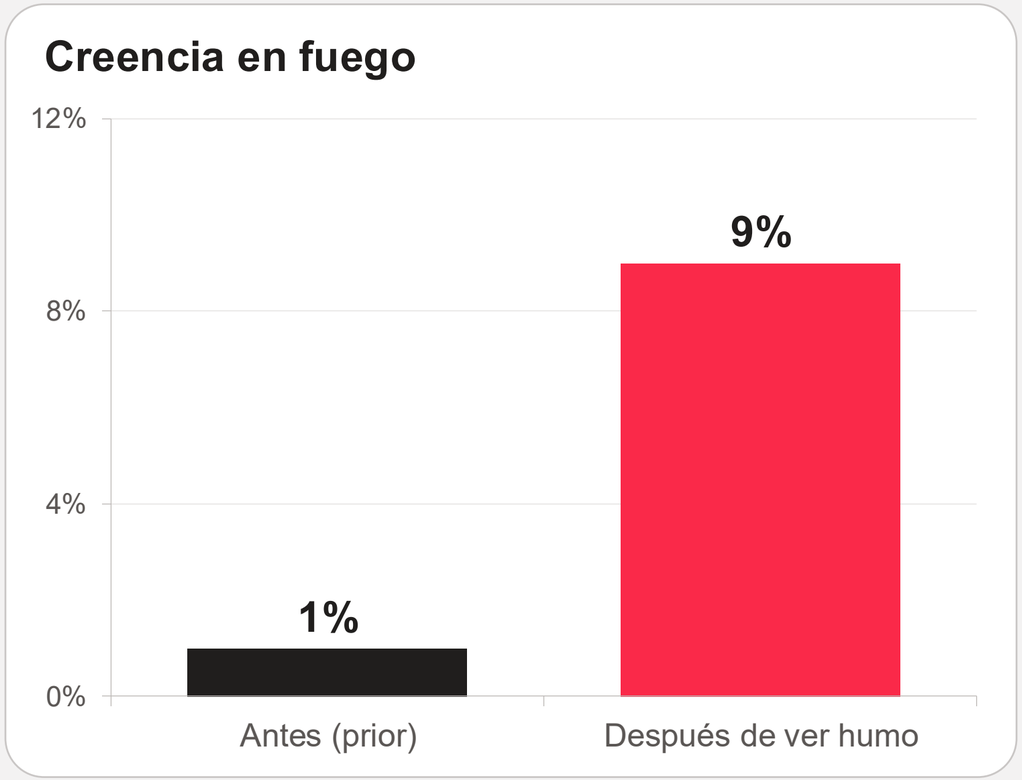
Naïve Bayes

Asume que las características son condicionalmente independientes dado y.

Cómo cambia la creencia con evidencia

Ejemplo: P(fuego)=1%, P(humo)=10%, P(humo|fuego)=90% → P(fuego|humo)=9%



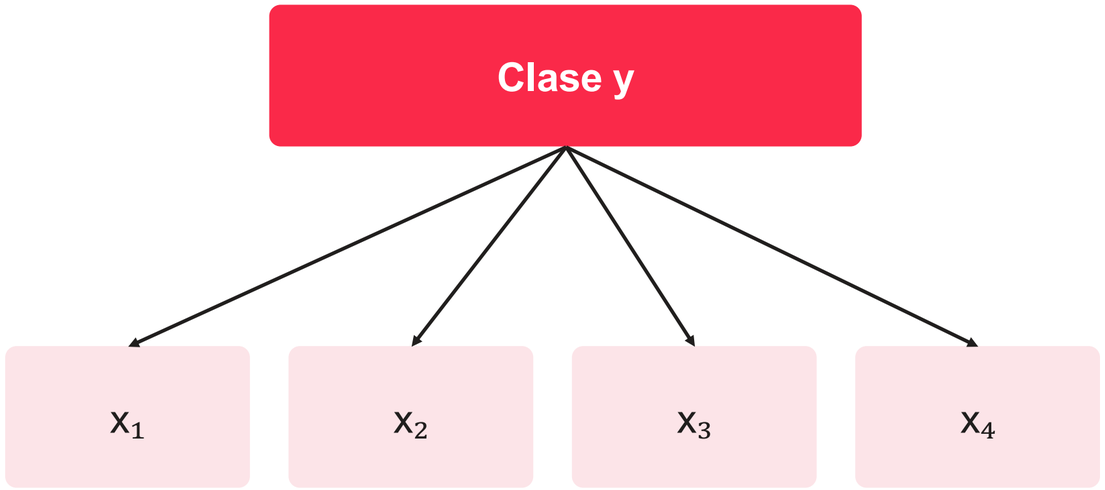


Entre 100 situaciones con humo, 9 tienen fuego.

La evidencia multiplicó por 9 la creencia inicial: de 1% a 9%.

Clasificador Naïve Bayes

$P(y \mid x_1 \ldots x_n) \propto P(y) \cdot \prod P(x_i \mid y)$



Cada característica depende solo de la clase (independencia condicional): no hay flechas entre las x_i.

1

Calcula el prior

P(y) de cada clase a partir de los datos.

2

Multiplica verosimilitudes

$\prod P(x_i|y)$, gracias a la independencia.

3

Elige la clase

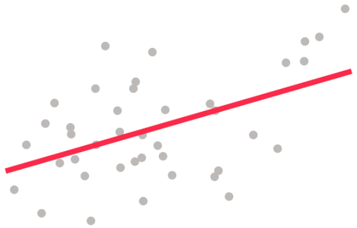
La de mayor posterior (MAP).

05

Análisis de regresión

05

Tipos de regresión



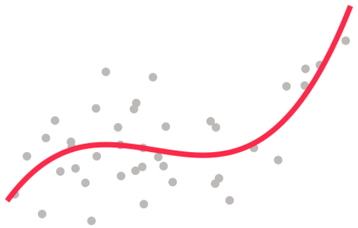
Lineal

Ajusta

Una línea

Error minimizado

Suma de error cuadrático medio



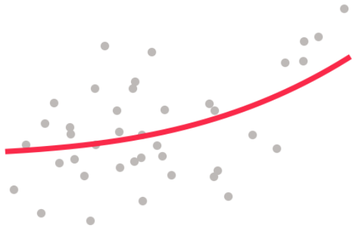
Polinomial

Ajusta

Un polinomio de grado k

Error minimizado

Suma de error cuadrático medio



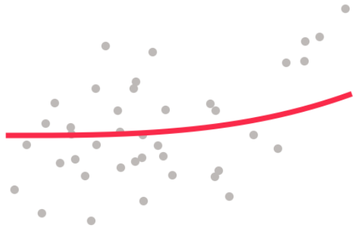
Ridge

Ajusta

Línea / polinomio

Error minimizado

Error cuadrático + norma L2 (β)



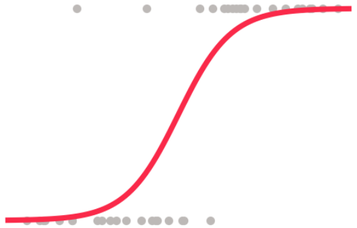
LASSO

Ajusta

Línea / polinomio

Error minimizado

Error cuadrático + norma L1 (β)



Logística

Ajusta

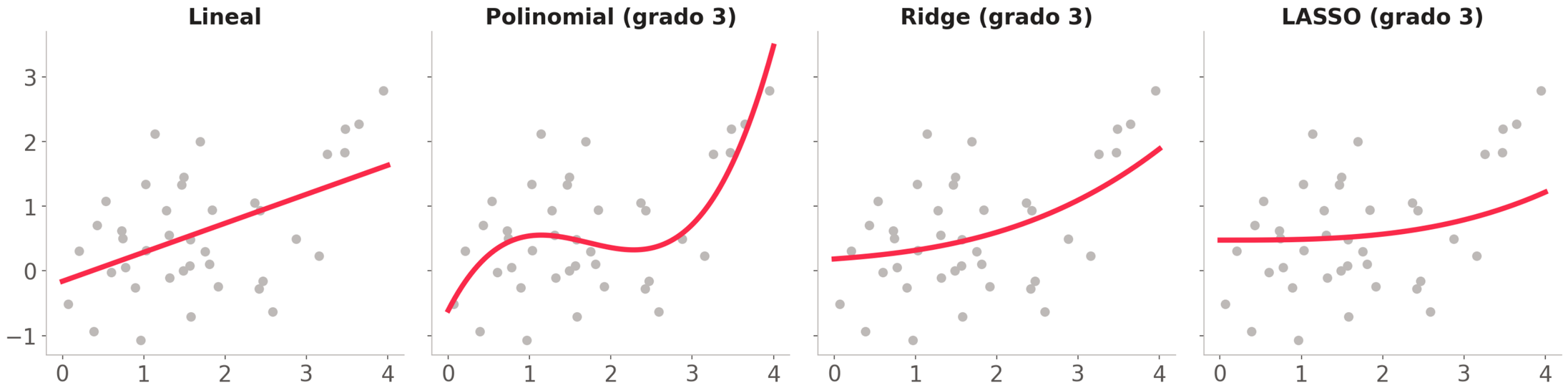
Línea / polinomio + sigmoide

Error minimizado

Entropía cruzada binaria

Ridge y LASSO se profundizan en el siguiente tema (Regularización).

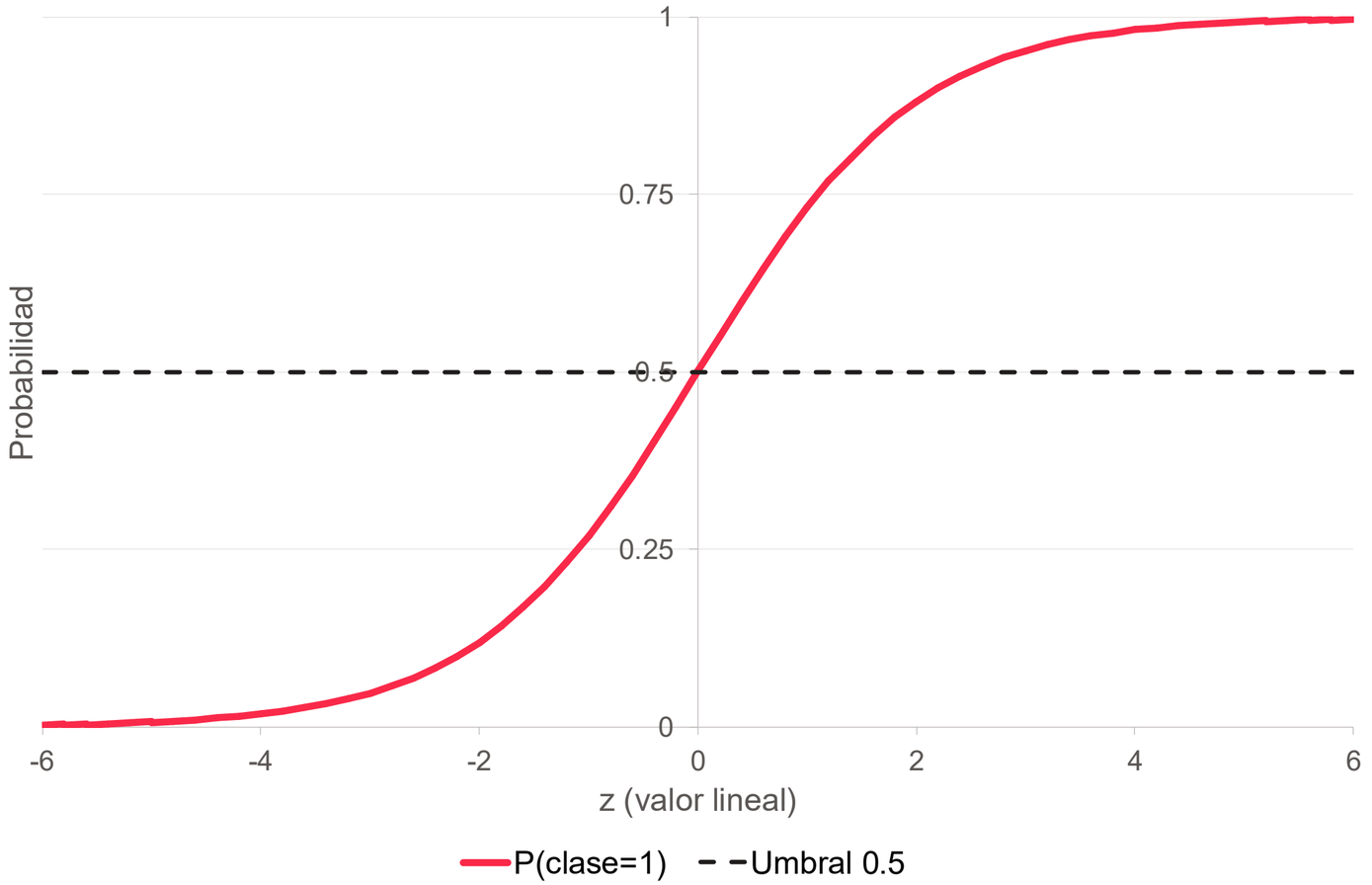
Comparación visual sobre los mismos datos



Mismos datos y mismo grado polinomial (salvo la lineal) para las cuatro curvas.

Ridge y LASSO producen curvas algo más suaves al penalizar coeficientes grandes.

Regresión logística: ajuste de una sigmoide



$\sigma(z) = 1 / (1 + e^{-z})$

La sigmoide convierte cualquier valor en una probabilidad entre 0 y 1.

Umbral de decisión

Típicamente 0.5 (línea punteada): ≥ 0.5 → clase 1; < 0.5 → clase 0.

06

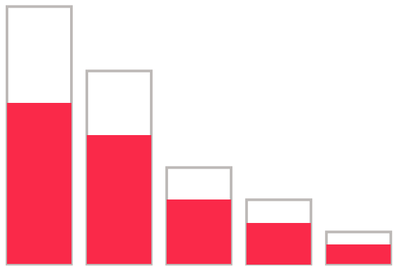
Regularización en ML

06

Reducir varianza a cambio de sesgo

L2 (Ridge)

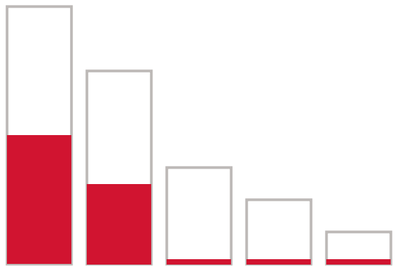
Penaliza el cuadrado de los coeficientes → los encoge, rara vez los anula.



gris = antes · color = después

L1 (LASSO)

Penaliza el valor absoluto → fuerza coeficientes a exactamente cero (selección automática de variables).



gris = antes · color = después

A mayor λ

Más fuerza de regularización → mayor sesgo, menor varianza.

K-fold CV

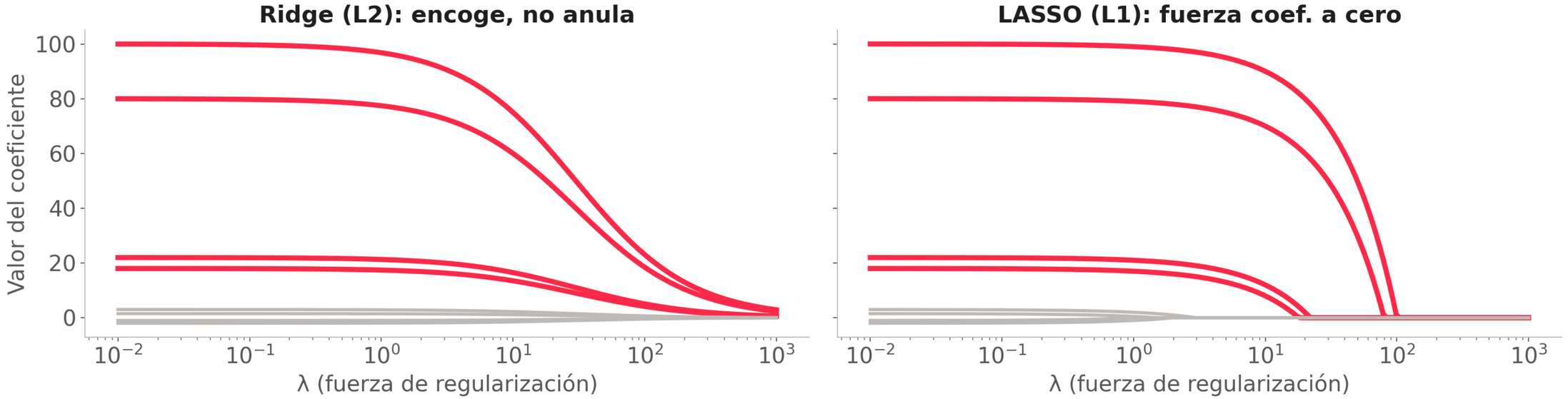
Divide en k grupos, entrena en k−1, evalúa en 1 y repite k veces.

Dropout

Desactiva aleatoriamente un % de conexiones en cada iteración de entrenamiento.

L1 vs. L2: el efecto sobre los coeficientes

Líneas rojas = variables informativas · líneas grises = ruido (trayectorias ilustrativas)

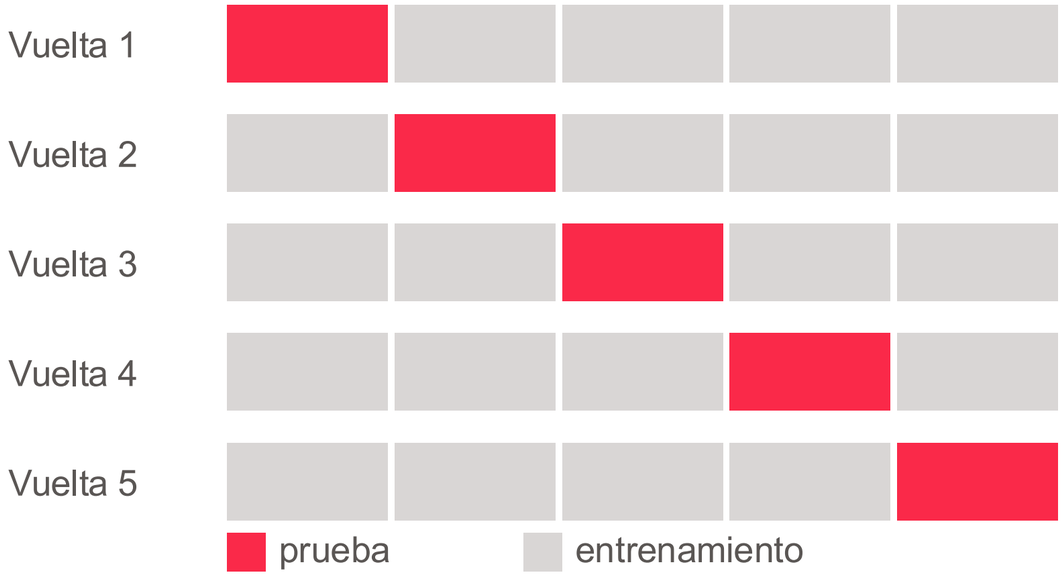


Ridge (L2): a mayor λ, encoge todos los coeficientes, casi nunca los anula.

LASSO (L1): varios coeficientes de ruido llegan a exactamente cero.

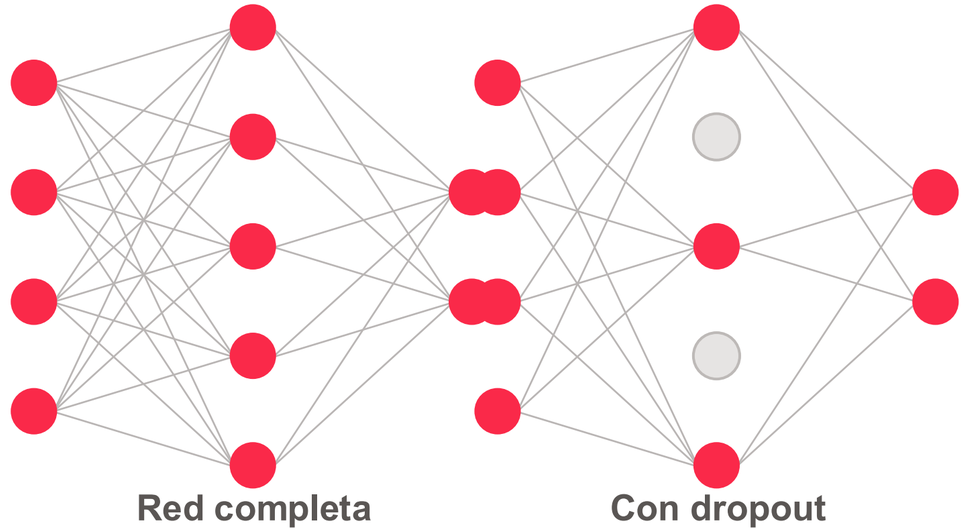
Validación cruzada y Dropout

Validación cruzada de 5 particiones



Cada partición sirve una vez como prueba: se usa todo el dataset para validar, sin dejar datos sin ver.

Dropout (tasa = 40%)



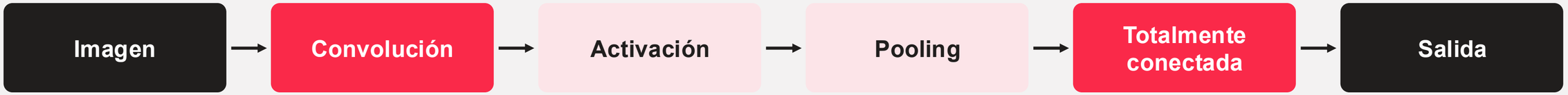
En cada iteración se apagan conexiones distintas al azar, evitando que la red dependa de unas pocas neuronas.

07

Redes Neuronales Convolucionales: Bloques Básicos

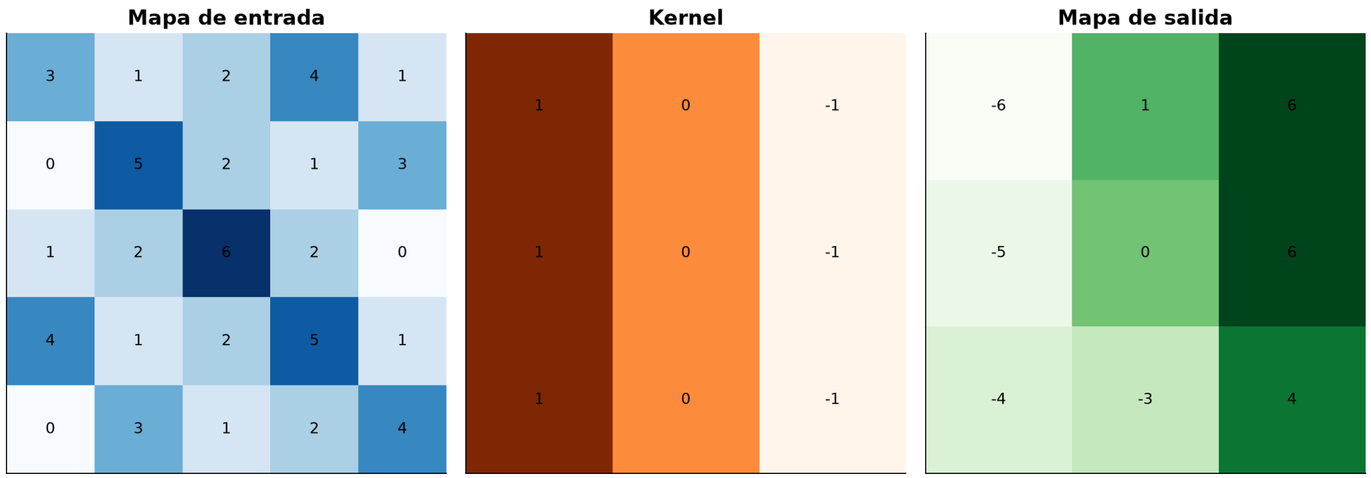
07

Los bloques de una CNN



Convolucional	**Entrenable**	Detecta patrones locales (bordes, texturas)
Totalmente conectada	**Entrenable**	Combina linealmente todas las entradas
Pooling (Max/Avg)	**No entrenable**	Reduce el tamaño espacial del mapa
Normalización	**Entrenable**	Estabiliza la escala de las activaciones
Activación	**No entrenable**	Introduce no-linealidad
Función de pérdida	—	Cuantifica el error de la predicción

La operación de convolución



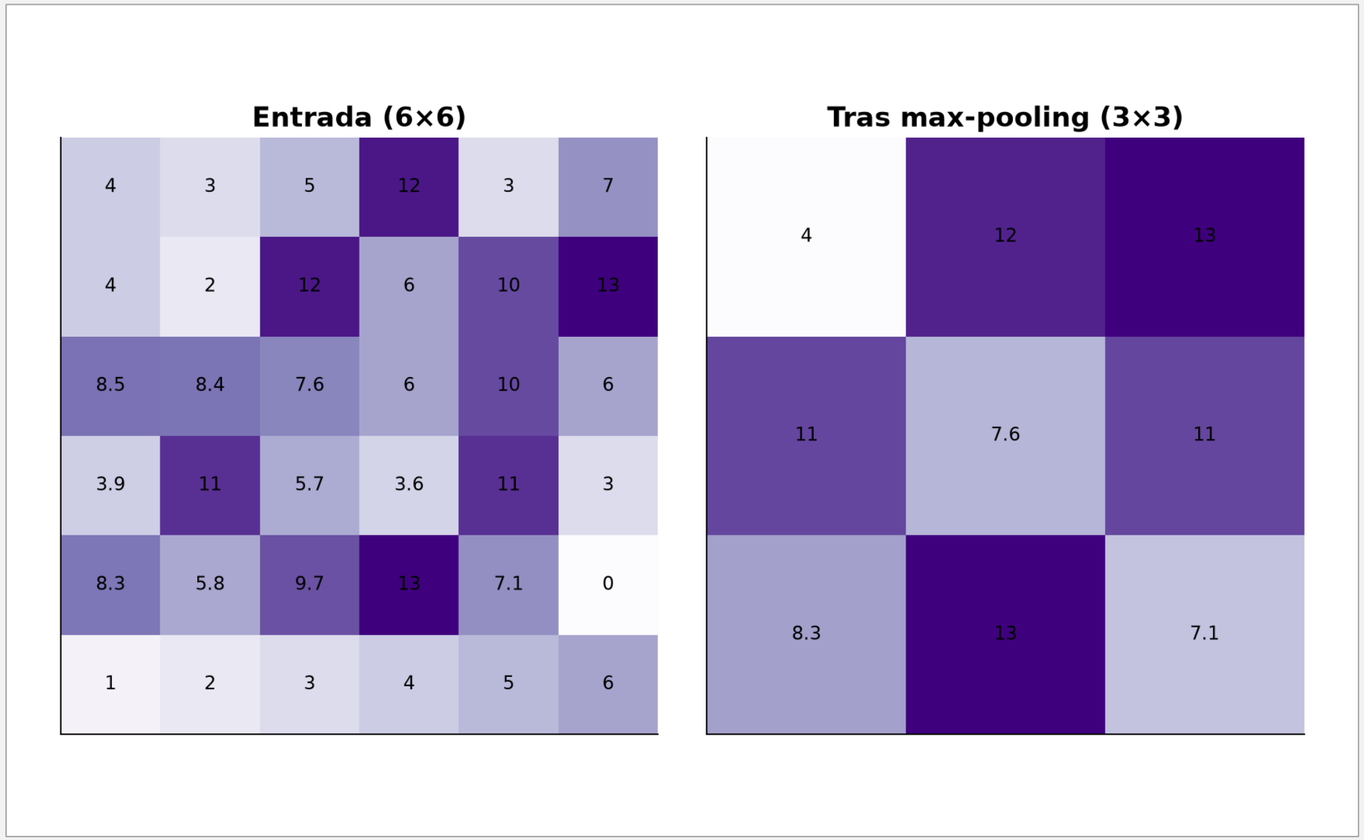
1. El kernel se desliza sobre la entrada calculando un producto punto en cada posición

2. Este kernel resalta bordes verticales (transiciones de claro a oscuro)

Max-Pooling

Conserva el valor máximo de cada ventana 2×2 — no tiene parámetros entrenables

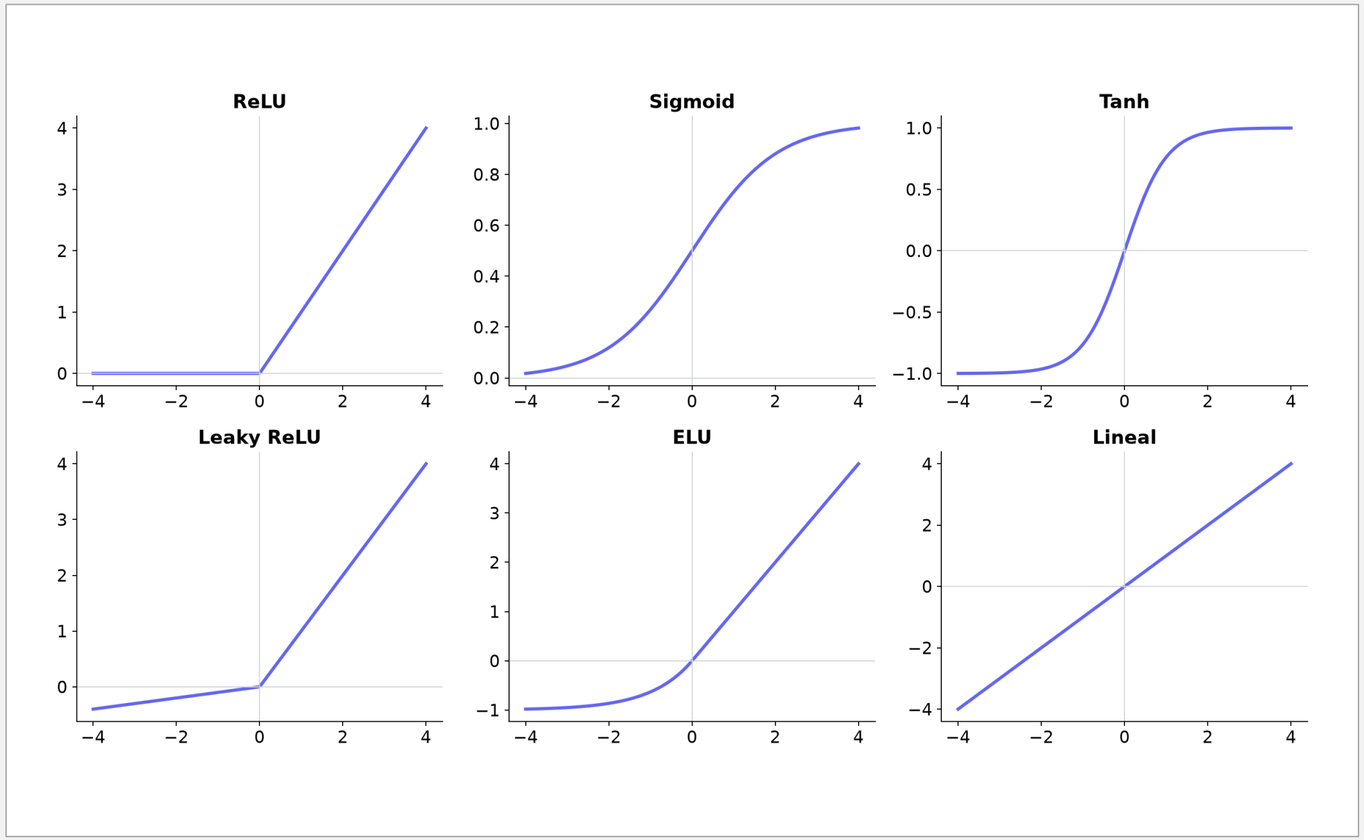
Reduce el tamaño espacial (aquí de 6×6 a 3×3) y con ello el cómputo de las capas siguientes



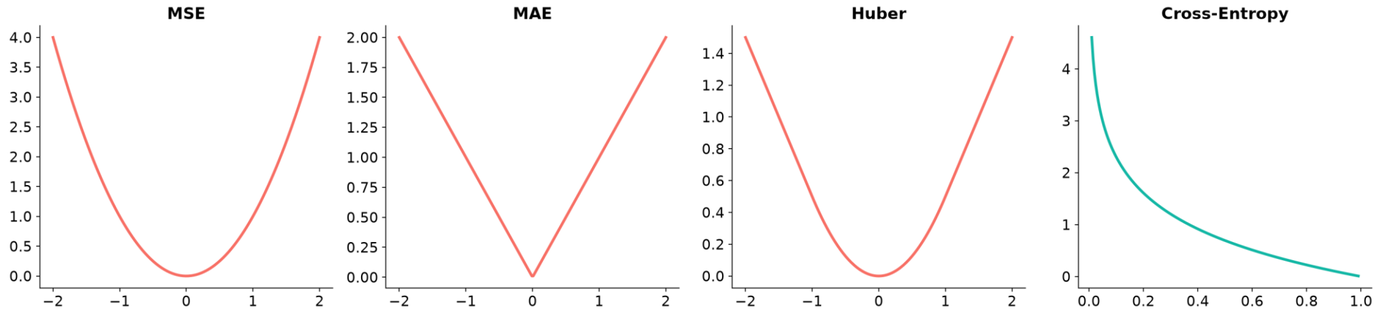
Funciones de activación

ReLU es la más usada en capas ocultas

Sigmoid/Tanh saturan en los extremos y se usan menos en redes profundas



Funciones de pérdida



1 MSE/MAE/Huber: para regresión — penalizan la distancia entre el valor real y el predicho

2 Cross-Entropy: para clasificación — penaliza según la probabilidad asignada a la clase correcta

08

CNNs Famosas

08

AlexNet, VGGNet, ResNet, Inception

2012
AlexNet

2014
VGGNet

2014
Inception

2015
ResNet

2012 · AlexNet

5 conv + 3 FC, ReLU — ganó el reto ImageNet

2014 · VGGNet

Kernels 3×3 uniformes — busca reducir parámetros

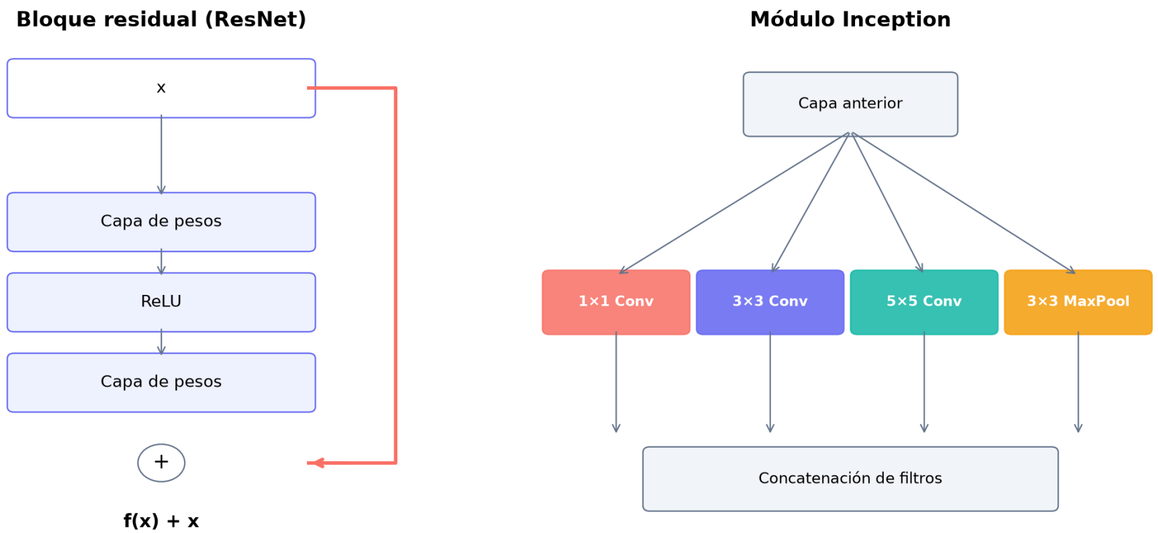
2015 · ResNet

Conexiones de atajo (skip connections) — resuelve el gradiente que desaparece en redes profundas

2014 · Inception / GoogLeNet

Varios tamaños de kernel en paralelo, concatenados — captura features a distintas escalas

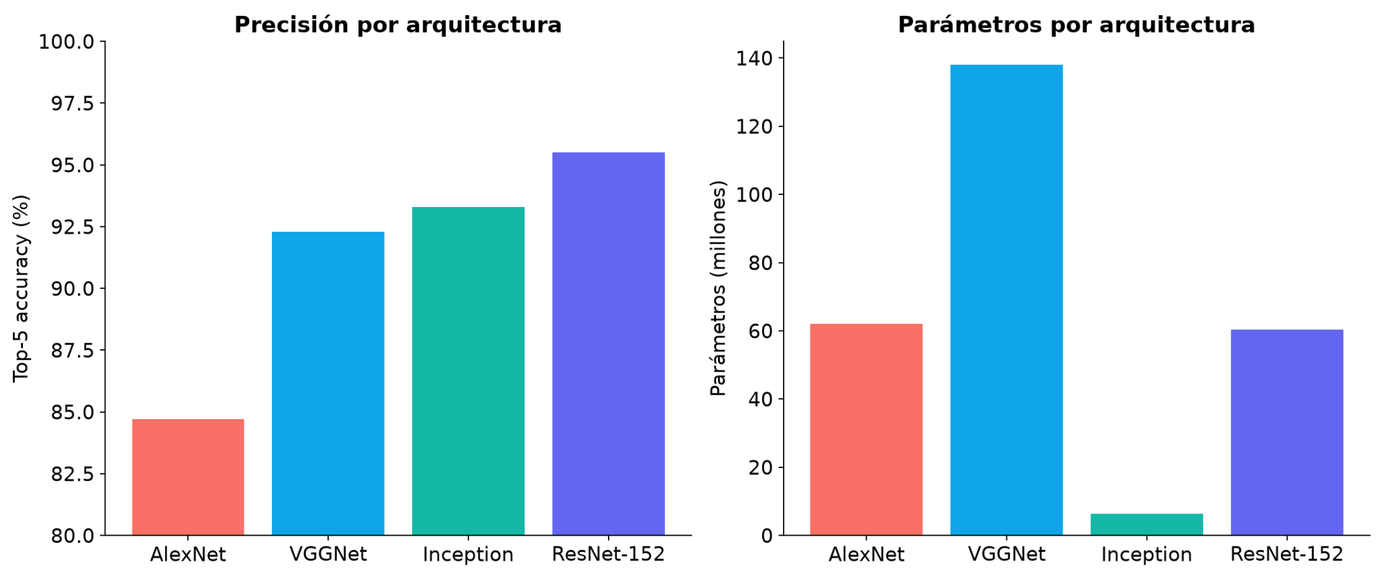
Bloque residual vs. módulo Inception



1. ResNet: la conexión de atajo deja que la capa aprenda solo el residuo f(x), facilitando entrenar redes muy profundas

2. Inception: aplica varios tamaños de kernel en paralelo y concatena sus salidas, capturando patrones a distintas escalas

Comparación de arquitecturas



1. Inception logra una precisión mayor que VGGNet con ~20 veces menos parámetros

2. Más parámetros no siempre significa mejor arquitectura — el diseño importa más que el tamaño

09

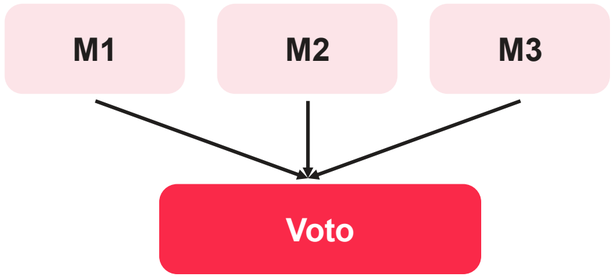
Aprendizaje por Ensamble

09

Bagging, Boosting, Stacking

Bagging

Paralelo



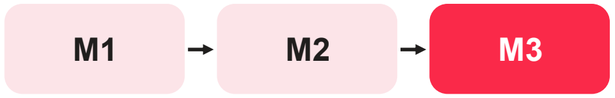
Se enfoca en
Reducir varianza

Modelos débiles
Homogéneos

Se combinan por
Votación simple

Boosting

Secuencial



Cada modelo corrige los errores del anterior

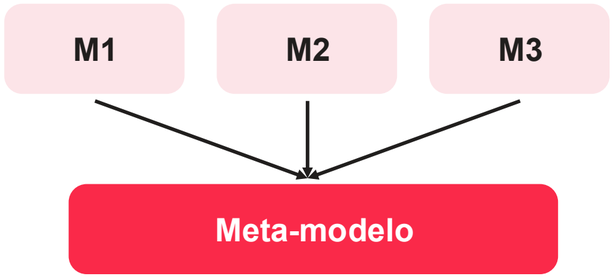
Se enfoca en
Reducir sesgo

Modelos débiles
Homogéneos

Se combinan por
Votación ponderada

Stacking

En capas

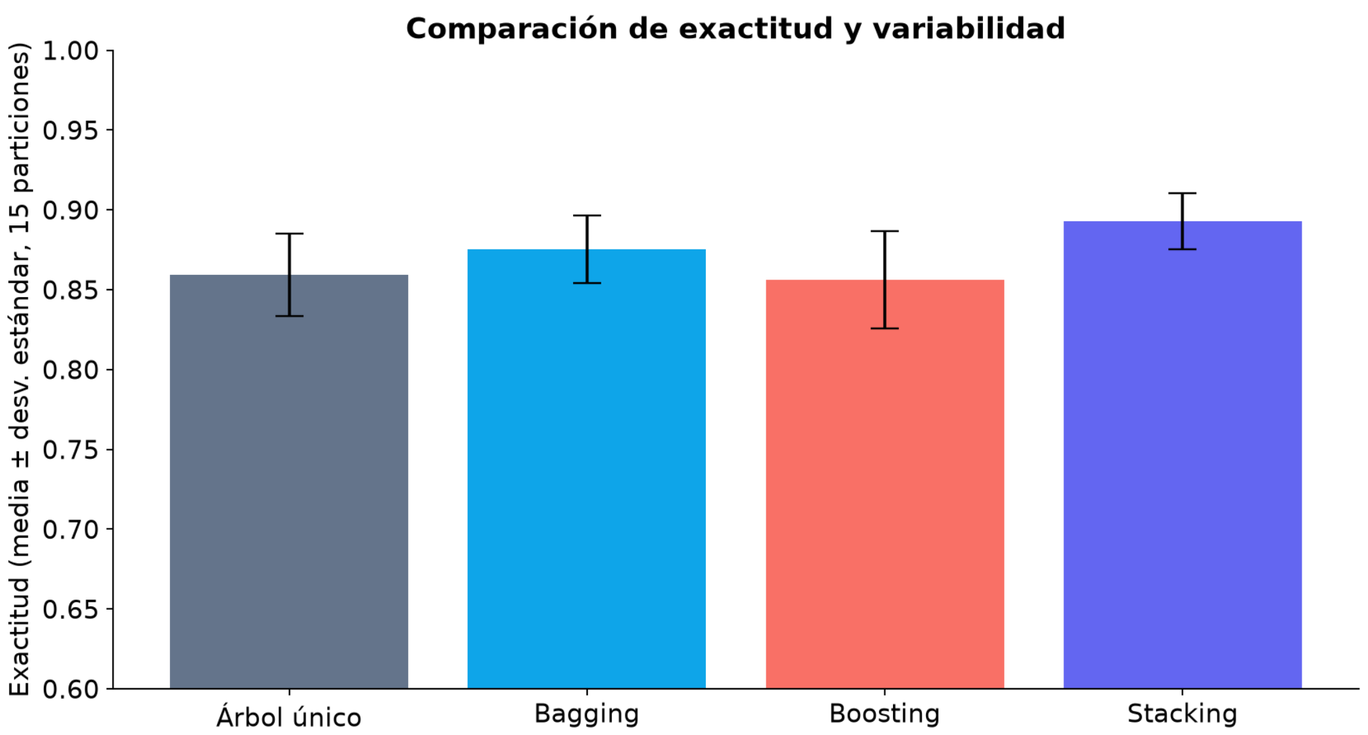


Se enfoca en
Mejorar exactitud

Modelos débiles
Heterogéneos

Se combinan por
Meta-modelo aprendido

Exactitud y variabilidad



1. Bagging reduce la variabilidad frente al árbol único, al promediar sobre 100 árboles distintos

2. Stacking combina la mayor exactitud promedio con la menor variabilidad de las cuatro opciones

10

Estructuras de Datos

10

Lista, Pila, Cola, Árbol, Grafo, Tabla Hash

Lista / Arreglo

Colección ordenada, acceso por índice

Pila (LIFO) y Cola (FIFO)

push/pop vs. enqueue/dequeue — la cola la usa BFS

Árbol y Grafo

Árbol: jerarquía sin ciclos · Grafo: vértices y aristas, puede tener ciclos

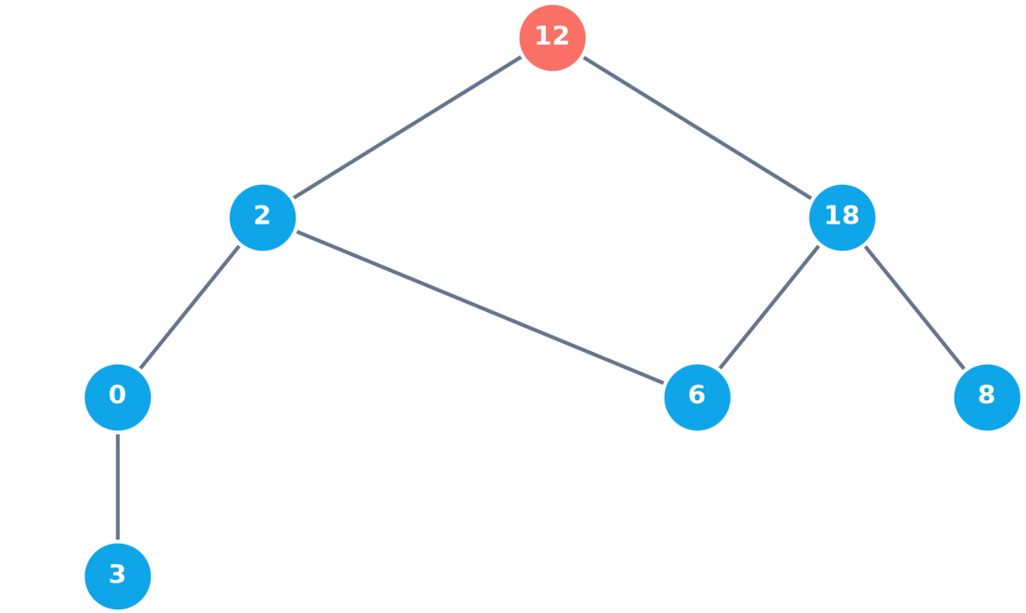
Tabla hash

Pares clave-valor, acceso en tiempo constante promedio

Complejidad típica

Estructura	Acceso	Búsqueda	Inserción
Lista / Arreglo	O(1)	O(n)	O(n)
Pila / Cola	O(n)	O(n)	O(1)
Tabla hash	—	O(1)	O(1)

BFS y DFS sobre un árbol



1. BFS explora nivel por nivel usando una cola (FIFO) · DFS explora en profundidad usando una pila o recursión

2. Mismos algoritmos usados en la búsqueda en espacios de estados (Unidad 2)

RECURSOS

¿Dónde profundizar cada tema?

Tema	Notebook del curso
Sesgo y varianza, sobre/subajuste	Unidad 4/1-fundamentos-redes-neuronales.ipynb
Métricas de clasificación	Unidad 3/6-pipelines-evaluacion.ipynb
PCA	Unidad 3/4-aprendizaje-no-supervisado.ipynb
Teorema de Bayes	Unidad 3/1-AI-fundamentals/3-teorema-bayes.ipynb
Regresión y regularización	Unidad 3/3-aprendizaje-supervisado/1-regresion-avanzada.ipynb
CNN y arquitecturas	Unidad 4/3-redes-convolucionales.ipynb
Ensambles	Unidad 3/5-modelos-ensamble.ipynb
Estructuras de datos, BFS/DFS	Unidad 2/1_buscar-estados-ia.ipynb